

Lesson 4: Fetch API — Pulling Live Data From the Web

Mission Brief

Right now the Lumexa Explorer only knows what its own onboard sensors tell it. But out there in the network are entire ground stations broadcasting real-time information — weather, star charts, telemetry — and today you learn to tune in. The **Fetch API** is how your JavaScript reaches across the internet to another server and brings real, live data back into your page.

Learning Objectives

By the end of this lesson, you will be able to:

- Explain what an API is and what JSON data looks like
- Use `fetch()` to request data from a real, public web API
- Use `.then()` to handle a Promise-based response and convert it to JSON
- Handle network errors and bad HTTP status codes gracefully
- Inspect real network requests using the browser's Network tab

What You'll Build

A "Ground Station Ping" tool: a button that, when clicked, fetches real, live data from the Open-Meteo weather API for a fixed location, and displays the current temperature on the page — your first real connection between JavaScript and the live internet, using `.then()` chains (async/await comes in Lesson 5).

Prerequisites

- Lesson 2 (DOM manipulation)
- Lesson 3 (event listeners)
- A basic understanding that data can be sent between computers over the internet

Key Concepts

- API (Application Programming Interface) and REST endpoints
- JSON (JavaScript Object Notation)
- `fetch(url)` and Promises
- `.then()`, `.catch()`, response `.ok` / `.status`
- `response.json()`
- The DevTools Network tab

Concept Explanation

An **API** (Application Programming Interface) is a defined way for one piece of software to request data or services from another. A **web API** typically means: you send an HTTP request to a URL (called an **endpoint**), and the server sends back data — usually formatted as **JSON** (JavaScript Object Notation), a text format that looks almost exactly like JavaScript objects and arrays: curly braces for objects, square brackets for arrays, `"key": value` pairs. JSON is the universal language APIs use to talk to JavaScript.

The **Fetch API** is the browser's built-in tool for making these requests. `fetch(url)` starts a network request and immediately returns a **Promise** — an object representing a value that isn't ready yet, but will be at some point (either successfully, or with an error). You attach `.then(callback)` to a Promise to say "once this finishes successfully, do this next." Fetch resolves its Promise with a `Response` object — but importantly, that response is not yet your actual data; it's more like an envelope. You call `.json()` on it (which *also* returns a Promise) to actually parse the body as JSON, which is why real Fetch code often chains two `.then()` calls: one to get the response, one to parse its body.

A crucial and often-missed detail: `fetch()`'s Promise only rejects (goes to `.catch()`) on a genuine network failure (no internet, DNS failure, CORS block). A "404 Not Found" or "500 Server Error" response is still a **successful** fetch as far as the Promise is concerned — the request completed, it just came back with a bad status. That's why real code always checks `response.ok` (a boolean shortcut for "status is in the 200-299 range") or `response.status` before trusting the data, and throws or handles an error manually if the check fails.

Errors can happen at any point in this chain: no internet connection, a typo in the URL, the API being down, or the server returning an error status for a bad request (like searching for a city that doesn't exist).

`.catch(errorHandler)` attached at the end of a `.then()` chain catches network-level failures and any error you deliberately `throw` inside an earlier `.then()`. Good API code always plans for the "it didn't work" case — never just the happy path.

Finally, the **Network tab** in DevTools lets you watch every request your page makes in real time — the exact URL, the response status, headers, and the raw JSON body — which is invaluable for confirming an API actually returned what you expected.

Guided Coding

```
<!-- index.html -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <title>Ground Station Ping</title>
  <style>
    body { font-family: sans-serif; background: #0b0e1a; color: #e6ecff; padding: 2rem; }
    #result { margin-top: 1rem; font-size: 1.2rem; }
    .error { color: #f87171; }
  </style>
</head>
<body>
  <h1>Ground Station Ping</h1>
  <p>Fetch live weather data for London using the free Open-Meteo API (no key required).</p>
  <button id="ping-btn">Ping Ground Station</button>
  <div id="result">No data yet.</div>
  <script src="script.js"></script>
</body>
</html>
```

```
// script.js

const pingButton = document.querySelector("#ping-btn");
const resultDiv = document.querySelector("#result");

// London's coordinates, hardcoded here for this simple lesson example.
// (In Lesson 5 and the Project 05 weather dashboard, coordinates come
// from a live geocoding search instead of being fixed like this.)
const LATITUDE = 51.5074;
const LONGITUDE = -0.1278;

const FORECAST_URL =
  `https://api.open-meteo.com/v1/forecast?latitude=${LATITUDE}&longitude=${LONGITUDE}` +
  `&current=temperature_2m,weather_code&timezone=auto`;

pingButton.addEventListener("click", () => {
  resultDiv.textContent = "Pinging ground station...";
  resultDiv.classList.remove("error");

  // fetch() returns a Promise. This is real network activity – open the
  // Network tab in DevTools and click the button to watch it happen live.
  fetch(FORECAST_URL)
    .then((response) => {
      // A fetch Promise resolving does NOT mean the request "succeeded" –
      // it only means we got a response at all. Always check .ok.
      if (!response.ok) {
        throw new Error(`Server responded with status ${response.status}`);
      }
      return response.json(); // parsing the body is ALSO asynchronous
    })
    .then((data) => {
      // At this point, "data" is a real JavaScript object parsed from
      // the API's JSON response.
      const temperature = data.current.temperature_2m;
      resultDiv.textContent = `Current temperature in London: ${temperature}°C`;
      console.log("Full API response:", data);
    })
    .catch((error) => {
      // Runs for network failures OR the error we threw above.
      resultDiv.textContent = `Ground station unreachable: ${error.message}`;
      resultDiv.classList.add("error");
      console.error("Fetch failed:", error);
    });
});
```

Code Walkthrough

- **FORECAST_URL**: built as a template literal string, embedding real latitude/longitude and requesting the `temperature_2m` and `weather_code` fields via the `current` query parameter — this is a real, working Open-Meteo endpoint that requires no API key at all.
- **The click handler**: immediately sets a "loading" message before the network call even starts, since `fetch` is asynchronous and the response won't arrive instantly.
- **First `.then()`**: receives the raw `Response` object. `response.ok` is `true` for any 2xx status. If it's `false`, the code deliberately `throw s` an `Error`, which skips every remaining `.then()` and jumps straight to `.catch()`.
- **`response.json()`**: parses the HTTP response body text into a real JavaScript object — this step is itself asynchronous (parsing can take time for large payloads), so it returns another Promise, which is why it's `return ed` for the next `.then()` to receive.
- **Second `.then()`**: receives the actual parsed `data` object. `data.current.temperature_2m` reaches into the nested JSON structure Open-Meteo returns (an object with a `current` key holding the requested fields) to pull out the specific value we want.
- **`.catch()`**: a single error handler catches both real network failures (no internet) and the manually thrown `Error` from the `.ok` check, showing a friendly message instead of leaving the user staring at a broken page.

Student Mission

1. Change the coordinates to your own city's approximate latitude/longitude and confirm you get a different, real temperature back.
2. Add `relative_humidity_2m` to the `current` query parameter in the URL, and display it alongside the temperature in `resultDiv`.
3. Open the Network tab, click the button, click on the request that appears, and take note of (in a code comment) what the "Preview" or "Response" tab shows — confirm it matches the `data` object you logged to the console.

Challenge

Deliberately break the URL (e.g., misspell `forecast` as `forcast`) and observe what happens — does it hit `.then()`'s `.ok` check, or does it fail differently? Then write code that also handles the case where `data.current` might be `undefined` (e.g., if the API changes its response shape), showing a fallback message like `"Unexpected response format"` instead of crashing with a `TypeError`.

Experiment/Extension

Try fetching from a URL that doesn't exist at all, like `https://api.open-meteo.com/v1/does-not-exist`, and observe the status code in your `.catch()` message versus what the Network tab shows. This helps you build intuition for the difference between a "the internet is down" failure and "the server responded, just with bad news."

Common Mistakes

- Assuming `fetch()` throws an error for 404/500 responses — it doesn't; you must check `response.ok` yourself.
- Forgetting that `response.json()` is itself asynchronous and needs its own `.then()` — trying to use the parsed data in the *same* `.then()` as the raw response.
- Building a URL with string concatenation and accidentally producing a malformed URL (missing `&`, extra spaces) — always double check the printed URL, e.g., with `console.log(FORECAST_URL)`.
- Not handling the `.catch()` case at all, leaving users with a silently broken button when their Wi-Fi drops.
- Hardcoding fake/sample data "just to see the UI work" and forgetting to swap in the real fetch call — always wire up the real network request from the start.

Debugging Tips

- Open the **Network tab**, filter by "Fetch/XHR", click your button, and inspect the request: check its **Status** column, then click it and check the **Response** or **Preview** tab to see the exact JSON returned.
- `console.log(FORECAST_URL)` before fetching to make sure the URL looks correct — you can even paste it directly into a new browser tab to see the raw JSON.
- If nothing happens on click, check the Console for a `TypeError` or `SyntaxError` first — a bug earlier in the `.then()` chain can silently swallow the rest.
- Use `console.error(error)` inside `.catch()` (not just a UI message) so you always have the technical detail available while debugging.

Check Your Understanding

1. What does `fetch()` return immediately, before any data has arrived?
2. Why do you need two separate `.then()` calls to get usable JSON data?
3. Why doesn't a 404 response trigger `.catch()` on its own?
4. What is JSON, and why do APIs commonly use it?
5. Where in DevTools can you see the raw response body of a fetch request?

Mini Quiz + Answer Key

- Q1.** What does `fetch(url)` return? A) The JSON data directly B) A Promise C) A string D) `undefined`
- Q2.** Which property tells you whether an HTTP response was successful (2xx)? A) `response.success` B) `response.ok` C) `response.good` D) `response.valid`
- Q3.** What does `response.json()` return? A) The data immediately B) Another Promise that resolves with the parsed data C) A string of raw text D) `null`
- Q4.** Where should you check for a bad HTTP status like 404? A) Fetch throws automatically, no check needed B) Manually, using `response.ok` or `response.status` C) Only in `.catch()` D) You cannot detect it

Answer Key: 1-B, 2-B, 3-B, 4-B

Lesson Recap

You learned what APIs and JSON are, how to make a real network request with `fetch()`, how to properly unwrap a `Response` into usable data with `.json()`, why checking `response.ok` matters, and how to catch and display errors — plus how to verify all of it using the DevTools Network tab.

Homework

Using the same Open-Meteo forecast endpoint, add a second button, "Ping 3-Day Outlook," that fetches with `&daily=temperature_2m_max,temperature_2m_min&forecast_days=3` added to the URL, and displays the three days' high/low temperatures as a simple list, handling errors the same way as the guided example.

Portfolio Project Connection

This lesson is the direct foundation of **Project 05, the Live Weather Dashboard** — the exact `fetch()` → check `.ok` → `.json()` → read fields pattern you practiced here is what powers the real geocoding and forecast calls in that project (refined further with `async / await` in Lesson 5).